Шаг 01. Репозиторий, окружение, зависимости

Блок 1. Основа разработки · Перед ним: 00. Подготовка · Следующий: 02. Чистый код

> **Время:** 6–9 часов (5–8 занятий). Обоснование — TIME.md.
> Незнакомые слова — словарь.

Зачем этот шаг

Любой коммерческий проект начинается не с кода, а с воспроизводимого окружения: коллега должен получить ровно те же версии библиотек, что у вас. Здесь закладывается скелет проекта, который будет расти все оставшиеся шаги.

Теория

- **Виртуальное окружение** — отдельная папка с библиотеками для одного проекта, чтобы зависимости разных проектов не конфликтовали между собой. Системный Python не трогаем никогда.
- **uv** — быстрый менеджер проектов и зависимостей, заменяющий связку `pip` + `venv` + `pip-tools`. Текущий стандарт для новых проектов.
- **pyproject.toml** — единый файл описания проекта: метаданные, зависимости, настройки инструментов. Заменил устаревший `setup.py`.
- **Backend сборки** — инструмент, который превращает папку с кодом в устанавливаемый пакет. Он указывается в `pyproject.toml`, но вы его не запускаете руками: это делает uv.
- **Lock-файл** — записанные точные версии всех зависимостей, включая транзитивные, то есть зависимости ваших зависимостей. Для повторяемого окружения также нужны согласованные версия Python, платформа и команда установки.
- **Runtime- и dev-зависимости** — первые нужны приложению при работе (например, будущий FastAPI), вторые только вам при разработке (линтер, pytest). Их держат в разных группах, чтобы dev-инструменты не попали в рабочий образ.
- **src-layout** — код пакета лежит в `src/<package>/`. Защищает от случайного импорта из рабочей директории вместо установленного пакета.
- **Git** — история изменений. Коммит фиксирует одно законченное изменение, а не «работу за день».

Что получится к концу шага и что из этого попадает в Git:

```
проект/
├── .github/workflows/        в Git     появится на шаге 2
├── src/имя_пакета/           в Git     ваш код
│   └── __init__.py
├── docs/project-scope.md     в Git     паспорт выбранной темы
├── pyproject.toml            в Git     зависимости и настройки инструментов
├── uv.lock                   в Git     точные версии, чтобы окружение повторялось
├── README.md                 в Git     что это, зачем, как запустить
├── .env.example              в Git     шаблон настроек без значений
├── .gitignore                в Git
├── .env                      НЕ в Git  настоящие секреты
└── .venv/                    НЕ в Git  окружение, восстанавливается из uv.lock
```

Как устроено виртуальное окружение

`.venv` — обычная папка: ссылка на интерпретатор и свой каталог `site-packages` для библиотек. «Активация» лишь ставит `.venv/bin` (на Windows — `.venv\Scripts`) первым в `PATH`. `uv run команда` выполняет команду в окружении без активации, предварительно сверив его с lock-файлом. Какой Python работает сейчас: `python -c "import sys; print(sys.executable)"`.

pyproject.toml по секциям

```
[project]
name = "job-tracker"
version = "0.1.0"
description = "Трекер поиска работы"
requires-python = ">=3.13"
dependencies = []                  # runtime-зависимости: сюда их допишет uv add

[project.scripts]
job-tracker = "job_tracker:main"   # команда в терминале -> функция в пакете

[dependency-groups]
dev = ["ruff>=0.6"]                # dev-зависимости: uv add --dev ruff

[build-system]                     # backend сборки; uv init выбирает его сам
requires = ["uv_build>=0.8,<0.9"]
build-backend = "uv_build"

[tool.ruff]                        # настройки инструментов живут в этом же файле
line-length = 100
```

В `pyproject.toml` записывают допустимый диапазон версий: `>=1.2` — не ниже, `~=1.2` — совместимые (не ниже 1.2, но меньше 2.0). Точную версию каждого пакета, включая транзитивные, выбирает uv и записывает в `uv.lock`. Файл зависимостей описывает намерение, lock-файл — результат. `requirements.txt` обычно плоский список без групп и без гарантии, что в нём перечислены все транзитивные пакеты; при необходимости его получают из lock-файла командой `uv export`.

Команда	Что делает
`uv init --package`	создаёт проект с src-layout и `pyproject.toml`
`uv add httpx` / `uv add --dev ruff`	добавляет зависимость, обновляет lock и окружение
`uv remove httpx`	удаляет зависимость
`uv lock`	пересчитывает lock-файл по `pyproject.toml`
`uv sync --locked`	приводит `.venv` к lock-файлу; ошибка, если lock устарел
`uv run pytest`	выполняет команду в окружении проекта
`uv python pin 3.13`	записывает версию Python в `.python-version`

Флаг `--locked` важен для CI и свежего клона: без него uv молча обновил бы lock-файл, и проверка прошла бы не на тех версиях, что закоммичены.

От какой ошибки защищает src-layout

При плоской раскладке пакет лежит в корне проекта рядом с тестами. Запуск из корня добавляет текущую папку в путь поиска модулей, и `import job_tracker` находит исходники в папке, а не установленный пакет. Тесты проходят, хотя установленный пакет может оказаться неполным:

например, в сборку не попал файл с данными. С `src/` импортировать пакет можно только после установки в окружение, поэтому вы проверяете то же, что получит пользователь.

Git: три области и ветки

```
рабочая папка ──git add──> индекс (staging) ──git commit──> история коммитов
     ^                                                            │
     └──────────── git switch ветка / git restore файл ───────────┘
```

Коммит — снимок всех файлов проекта плюс ссылка на родительский коммит, автор и сообщение. Индекс — черновик следующего коммита: `git add` кладёт туда только нужные изменения. Ветка — всего лишь подвижный указатель на коммит, `HEAD` — указатель на то, где вы сейчас. Поэтому создать ветку дёшево, а новый коммит сдвигает указатель текущей ветки вперёд.

Merge, rebase и конфликты

```
Исходно:   A──B──C          main
                \
                 D──E        feature

merge:     A──B──C──────M  main      M — коммит слияния с двумя родителями;
                \        /           история ветки сохранена как была
                 D──E──┘

rebase:    A──B──C──D'──E'  feature  коммиты пересозданы поверх C:
                                       у них новые хеши, история линейная
```

Rebase переписывает коммиты: если ветку уже скачал кто-то другой, его копия расходится с вашей. Отсюда правило — не переписывать опубликованную историю. Конфликт возникает, когда обе ветки изменили одни и те же строки, и Git не может выбрать сам:

```
<<<<<<< HEAD
Трекер откликов на вакансии
=======
Трекер поиска работы
>>>>>>> feature/readme
```

Вы оставляете нужный вариант или их сочетание, удаляете маркеры, выполняете `git add` и завершаете слияние `git commit`. Конфликт — не ошибка, а вопрос, на который отвечает человек.

Модели ветвления

- **GitHub Flow** — `main` всегда рабочий; на каждую задачу короткая ветка, PR, ревью и зелёный CI, затем слияние. Модель этой программы.
- **Trunk-based** — ветки живут часы, максимум день-два, и сливаются в общую основную очень часто; незаконченную функцию прячут за флагом в настройках, а не в ветке.
- **Git Flow** — долгоживущие `main` и `develop` плюс ветки feature, release и hotfix. Подходит продуктам с редкими нумерованными релизами, но тяжела для частой доставки.

Чем дольше живёт ветка, тем больше расходится с основной и тем тяжелее конфликты при слиянии, поэтому все современные модели сводятся к коротким веткам.

Какой коммит считается хорошим

Коммит содержит одно законченное логическое изменение, после которого проект работает. Первая строка сообщения — что сделано, в повелительном наклонении и коротко, до 50–72 символов; после

пустой строки — зачем, если это не очевидно:

```
Добавить загрузку записей из JSON

Демонстрационный скрипт читал данные из константы. Теперь путь к файлу
передаётся аргументом, кодировка UTF-8 задана явно.
```

Материалы

- uv: документация — разделы Getting started и Projects.
- Writing your pyproject.toml — что за секции в файле.
- src layout vs flat layout — зачем нужен `src/`.
- Pro Git, главы 1–3 — на русском, читать только основы работы с ветками.

Задание

1. Зафиксируйте тему в README, создайте `docs/project-scope.md` по паспорту. Добавьте первый issue и критерии по правилам работы.
2. Создайте репозиторий на GitHub, клонируйте локально. Основной CI использует GitHub Actions; GitLab будет отдельным обзорным приложением, переносить туда проект не потребуется.
3. Установите uv по официальной инструкции. Инициализируйте пакет через `uv init --package --python 3.13`, проверьте src-layout и backend сборки в `pyproject.toml`.
4. Пропишите в `pyproject.toml`: имя, описание, требуемую версию Python, точку входа.
5. Добавьте Ruff в dev-группу через `uv add --dev ruff`. Runtime-зависимости добавляйте по мере использования: первая понадобится на шаге 03.3. Зафиксируйте lock-файл и версию Python.
6. Настройте `.gitignore` (виртуальное окружение, кеши, `.env`, артефакты IDE).
7. Создайте `.env.example` — пока пустой шаблон с одной строкой-комментарием о назначении файла. Настоящий `.env` в репозиторий не попадает никогда; шаблон будет пополняться с шага 07.2.
8. Напишите README: что за проект, какую задачу решает, как поднять окружение и запустить.
9. Сделайте несколько осмысленных коммитов, работайте в отдельной ветке, слейте её через pull request.
10. Перенесите пример из подготовки в пакет и сохраните демонстрационный запуск. Подключите интерпретатор `.venv` в IDE. Проверьте импорт пакета из другой директории тем же интерпретатором.
11. В двух учебных ветках измените одну строку README, слейте их и разрешите конфликт. Объясните результат в issue. Ознакомьтесь с `venv` / `pip` и письменно сравните их с uv: кто создаёт окружение, кто устанавливает зависимости, что фиксирует lock-файл.

Обязательные критерии

- ☐ После установки Git и uv свежий клон проходит `uv sync --locked`; инструкция указывает версию Python.
- ☐ Lock-файл закоммичен, виртуальное окружение — нет.
- ☐ Код лежит в `src/`, пакет импортируется в интерпретаторе.
- ☐ Ruff находится в dev-группе.

- ▢ README отвечает на три вопроса: что это, зачем, как запустить.
- ▢ История содержит минимум три коммита с внятными сообщениями, работа влита через PR.
- ▢ Пакет импортируется вне рабочей папки, демонстрационный скрипт из подготовки работает.
- ▢ В репозитории есть `.env.example`, а `.env` попал в `.gitignore`.
- ▢ Паспорт определяет минимальный сценарий и расширения; issue связан с PR.

Желательные критерии

- ▢ Учебный конфликт в двух ветках разрешён, результат объяснён в issue.
- ▢ Написано сравнение uv с `venv` / `pip`: кто что делает и что фиксирует lock-файл.
- ▢ Ученик объясняет, куда добавлять будущие runtime-зависимости и почему они отделены от dev.

На что обратить внимание

- Не ставьте пакеты глобально через `pip install` — все зависимости только в проекте.
- `.env` не коммитится никогда; в репозитории живёт только `.env.example`.
- Сообщение коммита описывает, что изменилось и зачем, а не «fix», «update», «работа».

Вопросы для собеседования

Проверьте себя по темам этого шага. Ответов в программе нет намеренно: формулировка своими словами и есть проверка. Где можно — отвечайте на примере своего кода.

1. Зачем нужно виртуальное окружение? Что плохого в установке пакетов в системный Python?
2. Что делает uv и какие инструменты он заменяет?
3. Зачем нужен lock-файл, если версии уже указаны в `pyproject.toml`?
4. Чем lock-файл отличается от `requirements.txt`?
5. Что такое транзитивная зависимость и почему она попадает в lock-файл?
6. Зачем разделять runtime- и dev-зависимости? Что будет, если не разделять?
7. Что такое src-layout и от какой ошибки он защищает?
8. Что из этого попадает в Git, а что нет: `.venv`, `uv.lock`, `.env`, `.env.example`? Почему?
9. Что должно попадать в один коммит? Чем плохо сообщение «fix»?
10. Опишите путь изменения от issue до слияния в основную ветку.
11. Как возникает конфликт при слиянии и что вы делаете, когда его видите?
12. Чем отличается merge от rebase? Почему не переписывают опубликованную историю?
13. Что такое GitHub Flow и чем он отличается от trunk-based и Git Flow?

Полный набор по блоку — вопросы по блоку 1.